Web Technology Practical Assignments

Second Year MCA • Questions 1–13 with answers and execution instructions

Note: Save HTML programs with the .html extension and open them in a browser. For PHP programs, save files inside C:\xampp\htdocs\practical\, start Apache in XAMPP, and open them using http://localhost/practical/filename.php. Bootstrap, AngularJS and jQuery CDN examples need internet access.

Question 1: Create a static HTML page displaying the given output using frames.

Answer / procedure: The sample has a website heading, profile heading, left navigation links, and Name/Contact Number textboxes. Modern HTML no longer supports the old <frameset> element, so this code recreates the required frame-style layout using CSS.

```
<!DOCTYPE html>
<html>
<head>
<title>My Website</title>
<style>
body { font-family: "Times New Roman", serif; }
.container { width: 580px; height: 285px; border: 1px solid black; margin: 30px auto; }
.header { height: 53px; border-bottom: 1px solid black; padding: 5px 10px; }
.header h3 { margin: 0; }
.header p { text-align: center; margin: 2px 0; font-size: 18px; }
.content { display: flex; height: 225px; }
.menu { width: 190px; border-right: 1px solid black; padding: 15px 10px; }
.menu a { display: block; color: black; font-weight: bold; margin: 20px 0 40px; }
.details { padding: 45px 15px; }
.row { display: flex; align-items: center; gap: 10px; margin-bottom: 22px; font-size: 18px; }
input { width: 145px; height: 35px; border: 1px solid black; border-radius: 8px; }
</style>
</head>
<body>
<div class="container">
  <div class="header">
    <h3>MY WEBSITE</h3>
    <p>MY PROFILE</p>
  </div>
  <div class="content">
    <div class="menu">
      <a href="#home">Home</a>
      <a href="#profile">My Profile</a>
    </div>
    <div class="details">
      <div class="row"><label>Name: -</label><input type="text" name="name"></div>
      <div class="row"><label>Contact No.: -</label><input type="text" name="contact"></div>
    </div>
  </div>
</div>
</body>
</html>
```

Run: Save as question1.html (Save as type: All Files), then double-click it to open in Chrome.

Question 2: Design external CSS and link it to HTML: style headings, paragraphs with 13px font, text blue, and headings green.

Answer / procedure: Create two files in the same folder: question2.html and style.css.

File: question2.html

```
<!DOCTYPE html>
<html>
<head>
<title>External CSS Demo</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<h1>Welcome to My Website</h1>
<h2>About Us</h2>
<p>This paragraph uses a 13px font and blue text.</p>
</body>
</html>
```

File: style.css

```
h1, h2 {
    color: green;
}
p {
    font-size: 13px;
    color: blue;
}
```

Run: Save both files in the same folder and open question2.html in Chrome.

Question 3: Write JavaScript code for a Patient Master form and validate Patient ID, Patient Name, Address, City, Contact Number and Date of Birth.

Answer / procedure: This example checks required fields, numeric patient ID, 10-digit contact number, and a selected date.

```
<!DOCTYPE html>
<html>
<head><title>Patient Master</title></head>
<body>
<h2>Patient Master Form</h2>
<form onsubmit="return validateForm()">
  Patient ID: <input id="pid"><br><br>
  Patient Name: <input id="pname"><br><br>
  Address: <input id="address"><br><br>
  City: <input id="city"><br><br>
  Contact Number: <input id="contact" maxlength="10"><br><br>
  Date of Birth: <input id="dob" type="date"><br><br>
  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</form>
<script>
function validateForm() {
  let id = document.getElementById("pid").value.trim();
  let name = document.getElementById("pname").value.trim();
  let address = document.getElementById("address").value.trim();
  let city = document.getElementById("city").value.trim();
  let contact = document.getElementById("contact").value.trim();
  let dob = document.getElementById("dob").value;
```

```
    if (!id || !name || !address || !city || !contact || !dob) {
      alert("All fields are required.");
      return false;
    }
    if (!/^[0-9]+$/.test(id)) {
      alert("Patient ID must contain digits only.");
      return false;
    }
    if (!/^[A-Za-z ]+$/.test(name)) {
      alert("Patient name must contain letters and spaces only.");
      return false;
    }
    if (!/^[0-9]{10}$/.test(contact)) {
      alert("Contact number must be exactly 10 digits.");
      return false;
    }
    if (new Date(dob) > new Date()) {
      alert("Date of birth cannot be in the future.");
      return false;
    }
    alert("Patient form validated successfully.");
    return false;
  }
  </script>
  </body>
  </html>
```

Run: Save as question3.html and open in Chrome. This validates in the browser; it does not store records in a database.

Question 4: Create a Bootstrap grid with a 50% split on small, medium and large devices.

Answer / procedure: Bootstrap uses a 12-column grid. col-sm-6, col-md-6 and col-lg-6 each take 6 of 12 columns (50%) at their breakpoint.

```
<!DOCTYPE html>
<html>
<head>
<title>Bootstrap Grid</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container mt-4">
  <div class="row">
    <div class="col-6 col-sm-6 col-md-6 col-lg-6">
      <div class="p-3 border bg-light">Left Column (50%)</div>
    </div>
    <div class="col-6 col-sm-6 col-md-6 col-lg-6">
      <div class="p-3 border bg-secondary text-white">Right Column (50%)</div>
    </div>
  </div>
</div>
</body>
</html>
```

Run: Save as question4.html and open in a browser with internet access for the Bootstrap CDN.

Question 5: Design a responsive webpage using HTML, CSS and Bootstrap.

Answer / procedure: The following example uses a responsive navbar and Bootstrap grid.

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Website</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand" href="#">My Website</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#menu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#home">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#about">About</a></li>
        <li class="nav-item"><a class="nav-link" href="#contact">Contact</a></li>
      </ul>
    </div>
  </div>
</nav>
<div class="container my-4" id="home">
  <div class="p-4 bg-light rounded">
    <h1>Welcome</h1><p>This webpage adjusts to different screen sizes.</p>
  </div>
  <div class="row mt-3">
    <div class="col-12 col-md-6"><div class="p-3 border">Section One</div></div>
    <div class="col-12 col-md-6"><div class="p-3 border">Section Two</div></div>
  </div>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Question 6: Create a resume website in WordPress.

Answer / procedure: This is a practical setup task rather than a coding question. For a local installation, start Apache and MySQL in XAMPP, place WordPress in C:\xampp\htdocs\resume, create a database in phpMyAdmin, and complete the WordPress installer at http://localhost/resume.

Procedure:

1. Download and extract WordPress into C:\xampp\htdocs\resume.
2. Start Apache and MySQL in XAMPP Control Panel.
3. Open http://localhost/phpmyadmin and create a database named resume_db.
4. Open http://localhost/resume and follow the installation steps. Use the database name resume_db and local MySQL credentials configured for XAMPP.
5. Log in to WordPress, choose Appearance → Themes, install/activate a suitable resume or portfolio theme.
6. Create pages: Home, About Me, Education, Skills, Projects and Contact.
7. Set the navigation menu, add your resume information, preview the website and publish the pages.

Question 7: Build a navigation menu that highlights the selected entry using AngularJS directives.

Answer / procedure: This example uses ng-click and ng-class to highlight the selected menu item.

```
<!DOCTYPE html>
```

```
<html ng-app="menuApp">
<head>
<title>AngularJS Navigation</title>
<style>
.menu a { padding: 10px; display: inline-block; color: black; text-decoration: none; }
.active { background: green; color: white !important; }
</style>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="MenuController">
<h2>Navigation Menu</h2>
<div class="menu">
  <a href="" ng-click="selected='Home'" ng-class="{active:selected==='Home'}">Home</a>
  <a href="" ng-click="selected='About'" ng-class="{active:selected==='About'}">About</a>
  <a href="" ng-click="selected='Contact'" ng-class="{active:selected==='Contact'}">Contact</a>
</div>
<p>Selected: {{selected}}</p>
<script>
angular.module("menuApp", [])
.controller("MenuController", function($scope) {
  $scope.selected = "Home";
});
</script>
</body>
</html>
```

Run: Save as question7.html and open in Chrome with internet access for AngularJS.

Question 8: Create a Student Records application using AngularJS.

Answer / procedure: This basic example adds student records to an in-memory array. Data is lost when the page is refreshed; a database is needed for permanent storage.

```
<!DOCTYPE html>
<html ng-app="studentApp">
<head>
<title>Student Records</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body ng-controller="StudentController">
<h2>Student Records</h2>
<form ng-submit="addStudent()">
  Name: <input ng-model="newStudent.name" required>
  Roll No: <input ng-model="newStudent.roll" required>
  Class: <input ng-model="newStudent.class" required>
  <button type="submit">Add Student</button>
</form>
<br>
<table border="1" cellpadding="8">
<tr><th>Name</th><th>Roll No</th><th>Class</th><th>Action</th></tr>
<tr ng-repeat="s in students">
  <td>{{s.name}}</td><td>{{s.roll}}</td><td>{{s.class}}</td>
  <td><button ng-click="removeStudent($index)">Delete</button></td>
</tr>
</table>
<script>
angular.module("studentApp", [])
.controller("StudentController", function($scope) {
  $scope.students = [];
  $scope.newStudent = {};
  $scope.addStudent = function() {
    $scope.students.push(angular.copy($scope.newStudent));
    $scope.newStudent = {};
  };
```

```
  $scope.removeStudent = function(index) {
    $scope.students.splice(index, 1);
  };
});
</script>
</body>
</html>
```

Question 9: Write a jQuery program to get the value of a textbox.

Answer / procedure: The .val() method reads the current value from an input field.

```
<!DOCTYPE html>
<html>
<head>
<title>Get Textbox Value</title>
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
<h2>Get Textbox Value</h2>
Enter your name: <input type="text" id="username">
<button id="show">Show Value</button>
<p id="result"></p>
<script>
$(document).ready(function() {
  $("#show").click(function() {
    var value = $("#username").val();
    $("#result").text("Your name is: " + value);
  });
});
</script>
</body>
</html>
```

Run: Save as question9.html and open in Chrome with internet access for the jQuery CDN.

Question 10: PHP program to format user-entered text using chosen color, font and size, and save preferences in a cookie.

Answer / procedure: Save the following as question10.php. It displays the submitted text with selected styles and optionally stores style preferences in a cookie.

```
<?php
$color = $_POST['color'] ?? ($_COOKIE['color'] ?? 'blue');
$font = $_POST['font'] ?? ($_COOKIE['font'] ?? 'Arial');
$size = $_POST['size'] ?? ($_COOKIE['size'] ?? '16');
$text = $_POST['text'] ?? '';

$allowedColors = ['black','blue','red','green'];
$allowedFonts = ['Arial','Georgia','Times New Roman','Courier New'];
$allowedSizes = ['14','16','18','20','24'];

if (!in_array($color, $allowedColors)) $color = 'blue';
if (!in_array($font, $allowedFonts)) $font = 'Arial';
if (!in_array($size, $allowedSizes)) $size = '16';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['save'])) {
    setcookie('color', $color, time() + 30*24*60*60, '/');
    setcookie('font', $font, time() + 30*24*60*60, '/');
    setcookie('size', $size, time() + 30*24*60*60, '/');
}
```

```
?>
<!DOCTYPE html>
<html><head><title>Text Formatter</title></head><body>
<h2>Text Formatter</h2>
<form method="post">
  <textarea name="text" rows="5" cols="40" placeholder="Enter text"><?php
    echo htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
  ?></textarea><br><br>
  Color: <select name="color">
    <?php foreach (['black','blue','red','green'] as $c) {
      echo '<option '.($color===$c?'selected':'').' value="'.$c.'">'.$c.'</option>';
    } ?>
  </select>
  Font: <select name="font">
    <?php foreach (['Arial','Georgia','Times New Roman','Courier New'] as $f) {
      echo '<option '.($font===$f?'selected':'').' value="'.$f.'">'.$f.'</option>';
    } ?>
  </select>
  Size: <select name="size">
    <?php foreach (['14','16','18','20','24'] as $s) {
      echo '<option '.($size===$s?'selected':'').' value="'.$s.'">'.$s.'px</option>';
    } ?>
  </select>
  <label><input type="checkbox" name="save" value="yes"> Save preferences</label>
  <button type="submit">Format Text</button>
</form>
<?php if ($text !== '') { ?>
<h3>Formatted Output</h3>
<div style="color:<?php echo $color; ?>;font-family:<?php echo htmlspecialchars($font, ENT_QUOTES, 'UTF-8');
?>;font-size:<?php echo $size; ?>px;">
<?php echo nl2br(htmlspecialchars($text, ENT_QUOTES, 'UTF-8')); ?>
</div>
<?php } ?>
</body></html>
```

Note: Cookies must be set before any HTML output. The code sets cookies before the page markup. A cookie written on a POST request is normally available to the browser on the next request.

Question 11: Create a PHP form to capture Book Name, Author Name, Publisher Name, Category and Synopsis, with Reset and Submit buttons.

Answer / procedure: This version submits the form to the same PHP page and displays the captured values. It does not store them permanently in a database.

```
<!DOCTYPE html>
<html>
<head><title>Book Details</title></head>
<body>
<h2>Book Details Form</h2>
<form method="post">
  Book Name: <input type="text" name="book" required><br><br>
  Author Name: <input type="text" name="author" required><br><br>
  Publisher Name: <input type="text" name="publisher" required><br><br>
  Category: <input type="text" name="category" required><br><br>
  Synopsis:<br><textarea name="synopsis" rows="4" cols="35" required></textarea><br><br>
  <button type="reset">Reset</button>
  <button type="submit" name="submit">Submit</button>
</form>
<?php
if (isset($_POST['submit'])) {
  echo '<h3>Submitted Book Details</h3>';
  foreach (['book'=>'Book Name','author'=>'Author Name','publisher'=>'Publisher Name',
```

```
            'category'=>'Category','synopsis'=>'Synopsis'] as $key=>$label) {
    echo '<p><b>'.$label.':</b> '.nl2br(htmlspecialchars($_POST[$key] ?? '', ENT_QUOTES, 'UTF-8')).'</p>';
  }
}
?>
</body>
</html>
```

Run: Save as question11.php in the XAMPP practical folder and open through localhost. For permanent storage, add a MySQL table and INSERT query if your instructor requires it.

Question 12: Implement a PHP web service for a calculator application.

Answer / procedure: This version displays a calculator interface with two number fields and +, −, ×, ÷ buttons. PHP processes the selected operation and displays the result on the page. Save as q12.php in C:\xampp\htdocs and open http://localhost/q12.php with Apache running.

```
<?php
$result = ""; $error = "";
if (isset($_POST["calculate"])) {
    $a = $_POST["num1"] ?? ""; $b = $_POST["num2"] ?? ""; $op = $_POST["operation"] ?? "";
    if ($a === "" || $b === "" || !is_numeric($a) || !is_numeric($b)) { $error = "Please enter valid numbers."; }
    else {
        $a = (float)$a; $b = (float)$b;
        switch ($op) {
            case "add": $result = $a + $b; break;
            case "subtract": $result = $a - $b; break;
            case "multiply": $result = $a * $b; break;
            case "divide": if ($b == 0) { $error = "Cannot divide by zero."; } else { $result = $a / $b; } break;
            default: $error = "Select a valid operation.";
        }
    }
}
?>
<!DOCTYPE html><html><head><title>PHP Calculator</title>
<style>body{font-family:Arial;background:#eee}.calculator{width:320px;margin:60px
auto;padding:24px;background:white;border:1px solid #ccc;border-radius:10px;text-
align:center}input[type=number]{width:90%;padding:10px;margin:8px 0;box-sizing:border-
box}button{width:55px;padding:10px;margin:5px 2px;cursor:pointer}.result{margin-top:18px;color:green;font-
weight:bold}.error{margin-top:18px;color:red}</style></head><body>
<div class="calculator"><h2>PHP Calculator</h2><form method="post">
<input type="number" step="any" name="num1" placeholder="First number" value="<?php echo
htmlspecialchars($_POST['num1'] ?? ''); ?>" required>
<input type="number" step="any" name="num2" placeholder="Second number" value="<?php echo
htmlspecialchars($_POST['num2'] ?? ''); ?>" required><br>
<input type="hidden" name="operation" id="operation" value="add">
<button type="submit" name="calculate" value="1"
onclick="document.getElementById('operation').value='add'">+</button>
<button type="submit" name="calculate" value="1"
onclick="document.getElementById('operation').value='subtract'">−</button>
<button type="submit" name="calculate" value="1"
onclick="document.getElementById('operation').value='multiply'">×</button>
<button type="submit" name="calculate" value="1"
onclick="document.getElementById('operation').value='divide'">÷</button></form>
<?php if ($result !== "") { ?><div class="result">Result: <?php echo htmlspecialchars((string)$result);
?></div><?php } ?>
<?php if ($error !== "") { ?><div class="error"><?php echo htmlspecialchars($error); ?></div><?php } ?>
</div></body></html>
```

Run: Save as q12.php in C:\xampp\htdocs, start Apache, then open http://localhost/q12.php. MySQL is not required for Q12.

Question 13: Connect PHP to MySQL and store book details in a database.

Answer / procedure: This additional practice question extends Q11. Create a database and table in phpMyAdmin, then connect PHP to MySQL and insert submitted book details. Start Apache and MySQL in XAMPP.

Step 1 — Create the database and table

Open http://localhost/phpmyadmin, click SQL, and run the following commands. If college_db already exists, do not run CREATE DATABASE again; select it and create the table only if needed.

```
CREATE DATABASE college_db;
USE college_db;
CREATE TABLE books (
    id INT AUTO_INCREMENT PRIMARY KEY,
    book_name VARCHAR(100) NOT NULL,
    author_name VARCHAR(100) NOT NULL,
    publisher_name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    synopsis TEXT NOT NULL
);
```

Step 2 — Save this program as q13.php in C:\xampp\htdocs\

```
<?php
$conn = new mysqli("localhost", "root", "", "college_db");
if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); }
$message = "";
if (isset($_POST["submit"])) {
    $book = trim($_POST["book"] ?? ""); $author = trim($_POST["author"] ?? "");
    $publisher = trim($_POST["publisher"] ?? ""); $category = trim($_POST["category"] ?? "");
    $synopsis = trim($_POST["synopsis"] ?? "");
    if ($book && $author && $publisher && $category && $synopsis) {
        $stmt = $conn->prepare("INSERT INTO books (book_name, author_name, publisher_name, category, synopsis)
VALUES (?, ?, ?, ?, ?)");
        $stmt->bind_param("sssss", $book, $author, $publisher, $category, $synopsis);
        $message = $stmt->execute() ? "Book saved successfully!" : "Error saving book."; $stmt->close();
    } else { $message = "Please fill in all fields."; }
}
?>
<!DOCTYPE html><html><head><title>Book Details</title></head><body><h2>Book Details Form</h2>
<form method="post">Book Name: <input name="book" required><br><br>
Author Name: <input name="author" required><br><br>
Publisher Name: <input name="publisher" required><br><br>
Category: <input name="category" required><br><br>
Synopsis: <textarea name="synopsis" required></textarea><br><br>
<button type="reset">Reset</button> <button type="submit" name="submit">Submit</button></form>
<p><?php echo htmlspecialchars($message); ?></p></body></html>
<?php $conn->close(); ?>
```

Step 3 — Run and verify

Save as C:\xampp\htdocs\q13.php. Start Apache and MySQL, then open http://localhost/q13.php. Submit a book record. Verify it in phpMyAdmin → college_db → books → Browse, or run SELECT * FROM books; in the SQL tab.

Connection note: This assumes MySQL is on localhost port 3306 with username root and a blank password. If your MySQL uses a password or a different port, update the connection settings.

Quick Revision: How to Run Each Program

- Q1, Q3, Q4, Q5, Q7, Q8 and Q9: save as .html and open in Chrome.
- Q2: save question2.html and style.css in the same folder; open the HTML file.
- Q6: install WordPress locally using XAMPP and a MySQL/MariaDB database.

- Q10, Q11 and Q12: save .php files in C:\xampp\htdocs\, start Apache, and open through http://localhost/filename.php. Q13 additionally requires MySQL and a database/table.
- If Bootstrap, AngularJS or jQuery does not load, check internet connectivity because these examples use CDN links.
- Database storage is not included in Q3 and Q8 because the supplied questions do not explicitly require it. Q13 demonstrates persistent storage using PHP and MySQL.

Important Exam Tips

- Use Save as type: All Files in Notepad to avoid accidentally saving files as .txt.
- PHP code will not execute by double-clicking the file; use localhost through XAMPP/WAMP.
- The Reset button clears the form controls; it does not delete records already stored in a database.
- For production systems, validate data on the server as well as in the browser and use prepared SQL statements for database operations.

Additional Notes: WordPress Resume Website Setup (Q6)

Aim: To create a two-page resume website using WordPress, PHP, MySQL and XAMPP.

Procedure

8. **Download WordPress:** Visit https://wordpress.org/download/ and download the WordPress ZIP file. Extract it.
9. **Create the website folder:** Open C:\xampp\htdocs, create a folder named resume, then copy all files and folders inside the extracted wordpress folder into C:\xampp\htdocs\resume. Ensure index.php is directly inside the resume folder.
10. **Start XAMPP:** Open XAMPP Control Panel and start Apache and MySQL. If MySQL fails because port 3306 is occupied, resolve the port conflict before continuing.
11. **Create the database:** Open http://localhost/phpmyadmin, click New, enter resume_db, and click Create.
12. **Run the installer:** Open http://localhost/resume. Select a language and enter the database name resume_db, username root, password blank only if the XAMPP root account has no password, database host localhost, and table prefix wp_. Click Submit and continue the installation.
13. **Set up the administrator:** Enter the site title My Resume Website, choose an administrator username and strong password, enter a valid email address, and click Install WordPress. Store the login details securely.
14. **Log in:** Open http://localhost/resume/wp-admin and log in with the administrator credentials.
15. **Create two pages:** Go to Pages → Add New Page. Create and publish Home (introduction, profile photo and short professional summary) and Resume (education, technical skills, projects and contact details).
16. **Choose a design:** Go to Appearance → Themes and activate a suitable theme. Use Appearance → Editor if available to customize the design and navigation.
17. **Add navigation and test:** Add Home and Resume links to the site navigation, publish the pages, and visit http://localhost/resume to check the website.

Result

A two-page resume website is created locally using WordPress and XAMPP.